

ОТЧЕТ

о выполнении практической работы №2.3

Дисциплина «Оптимизация веб-приложений»

Тема: «Модель на PHP: работа с базой данных»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Чеголин Алексей Михайлович

Студент группы: 4ИСИП-923

Проверил:

Преподаватель: Рыбаков А.С.

Дата: «29 сентября 2026 г.»

Оценка: _____

Теоретическая часть:

Модель — слой приложения, который отвечает за данные: их структуру, правила обработки и доступ к базе. Модель реагирует на команды контроллера, возвращает или изменяет своё состояние и передаёт данные в представление. Модель не зависит от представления и контроллера и не может использовать их классы.

Главный принцип, который вводится этой практикой: SQL-запросы должны жить только в модели. Контроллер не должен знать, откуда берутся данные, — из базы, файла или внешнего API. Если источник данных изменится, правки потребуются только в модели.

Ход работы:

Задание 1. Создание базы данных

Создана база `mvc_demo` в кодировке `utf8mb4` и таблица `articles` из четырёх полей. Тестовые данные добавлены тремя записями — теми же, что использовались в предыдущей практике.

Поле	Тип	Назначение
<code>id</code>	<code>INT AUTO_INCREMENT PRIMARY KEY</code>	первичный ключ, уникальный номер статьи
<code>title</code>	<code>VARCHAR(255) NOT NULL</code>	заголовок статьи
<code>content</code>	<code>TEXT NOT NULL</code>	текст статьи
<code>created_at</code>	<code>TIMESTAMP DEFAULT CURRENT_TIMESTAMP</code>	дата создания, подставляется автоматически

Скрипт создания базы сохранён в проекте файлом `schema.sql`, чтобы структуру можно было воспроизвести на любой машине одной командой.

Задание 2. Единое подключение и модель `Article`

Создан класс `Database`, который хранит единственное соединение PDO на всё приложение и отдаёт его по запросу. Одно соединение вместо нового на каждый запрос экономит ресурсы сервера базы данных. Установлены режим сообщения об ошибках через исключения и ассоциативный способ получения строк, а также отключена эмуляция подготовленных выражений — запросы уходят в MySQL в подготовленном виде.

Класс `Article` содержит методы `all()` для получения списка статей и `find()` для одной статьи по номеру. Оба запроса построены на подготовленных выражениях: значение подставляется параметром, а не склеивается со строкой запроса, поэтому SQL-инъекция невозможна.

Контроллер изменился минимально: он по-прежнему передаёт в представление массив, но берёт его не из собственного кода, а у модели. Форма вызова представления не изменилась — поменялся только источник данных, что и было целью работы.

MVC Blog

Все статьи

Первая статья

[Читать →](#)

Создана: 2026-09-29 23:22:22

Вторая статья

[Читать →](#)

Создана: 2026-09-29 23:22:22

Третья статья

[Читать →](#)

Создана: 2026-09-29 23:22:22

Рис. 1 – Список статей из базы данных: дата создания подставляется из поля `created_at`

MVC Blog

[← Назад](#)

Вторая статья

Здесь мы разбираем контроллеры.

Создана: 2026-09-29 23:22:22

Рис. 2 – Страница статьи: данные получены методом `find()` по номеру из адреса

Обработка отсутствующей записи

Метод `find()` возвращает `null`, если записи с таким номером нет. Контроллер проверяет результат и отдаёт код 404 с сообщением вместо пустой страницы — это разбирается и в практике 2.4.

Статья не найдена

Рис. 3 – Запрос несуществующей статьи `/article/999`: контроллер отдаёт код 404

Ответы на вопросы практики:

1. Контроллер не изменил форму вызова представления — он всё так же отдаёт массив. Меняется только источник данных.

Именно это и подтвердилось: в контроллере изменилась одна строка получения данных, а шаблон и общий каркас остались без правок.

2. Модель — единственное место, где живёт SQL.

Запросы к таблице `articles` находятся только в классе `Article`. Ни контроллер, ни шаблоны не обращаются к базе: контроллер вызывает методы модели, шаблон работает с готовым массивом.

3. Если завтра статьи будут в JSON-файле или в API — меняется только модель, контроллер и `view` не трогаются.

Это и есть практический смысл разделения: интерфейс методов модели (`all` и `find`) можно сохранить, а их наполнение заменить, не затрагивая остальные слои приложения.

Примечание: в файле задания «Задание 1» и «Задание 2» содержат одинаковый материал — одну и ту же схему базы и один и тот же код. Работа выполнена один раз и охватывает оба пункта.

Вывод:

Создана база данных `mvc_demo` и таблица `articles`, реализован слой моделей с единым подключением к базе через PDO. Все запросы собраны в одном классе, значения передаются параметрами, поэтому SQL-инъекция невозможна. Контроллер при этом не изменил форму работы с представлением — поменялся только источник данных, что наглядно подтверждает главный принцип MVC: каждый слой можно заменить независимо от остальных.

Отдельно отработана обработка отсутствующей записи: метод модели возвращает `null`, а контроллер превращает это в корректный ответ 404 вместо пустой страницы.